# Why No One Pwned Synology at Pwn2Own and Tianfu Cup in 2021

## Analyzing Defensive Coding Techniques from a Vulnerability Researcher's Perspective

# whoarewe

**Eugene Lim “spaceraccoon”**

- Research and Innovation @ Government Technology Agency of Singapore
- Disclosed code execution vulnerabilities in Apache OpenOffice (CVE-2021-33035) and Microsoft Office (CVE-2021-38646)
- AI-powered phishing @ DEF CON 29 & Black Hat USA 2021

# whoarewe

**Loke Hui Yi “angelystor”**

- Research and Innovation @ Government Technology Agency of Singapore
- Number 1 hit when you google for “winafl fuzzing tutorial”
- Hunting application backdoors @ Black Hat Asia 2020

# What we'll be talking about

## Background

Pwn2Own and TianFu Cup

## Technical Review

Synology DiskStation Manager

## Defensive Coding 1

Declarative Authentication and Authorization

## Defensive Coding 2

Hardened Standard Library Calls

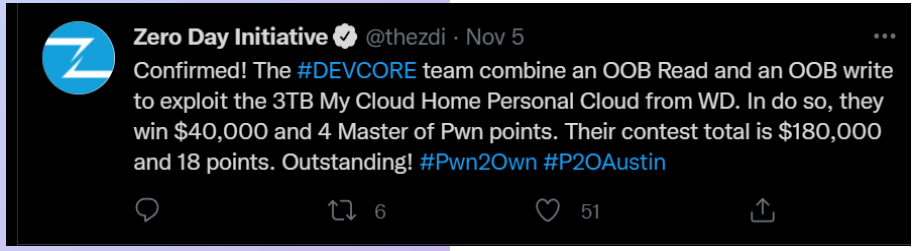
## Defensive Coding 3

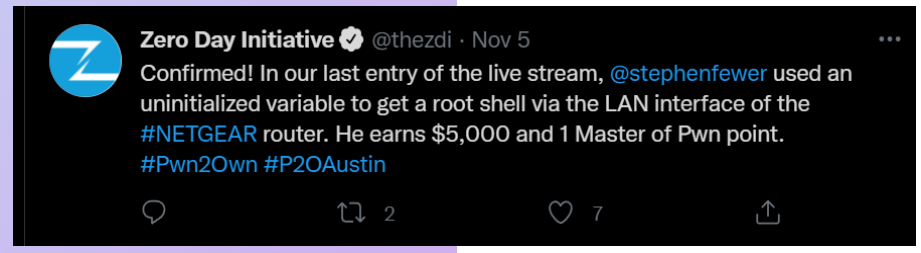
Shared Reusable Application Functions

# Background

# 2021's zero-day contests saw record-breaking numbers.

- **Windows 10** – hacked 5 times
- **Adobe PDF Reader** – 4 times
- **Ubuntu 20** – 4 times
- **Parallels VM** – 3 times
- **iOS 15** – 3 times
- **Apple Safari** – 2 times
- **Google Chrome** – 2 times
- **ASUS AX56U router** – 2 times
- **Docker CE** – 1 time
- **VMWare ESXi** – 1 time
- **VMWare Workstation** – 1 time
- **qemu VM** – 1 time
- **Microsoft Exchange** – 1 time




Source: The Record

# Despite featuring in both contests, Synology's devices were un-pwned.

The only ones that were not hacked included Synology DS220j NAS, Xiaomi Mi 11 smartphone, and a Chinese electric vehicle whose brand was never revealed — for which no participant even registered to attempt an exploit.

Source: The Record

| Target | Prize | Master of Pwn Points |
|---|---|---|
| Synology DiskStation DS920+ | $40,000 (USD) | 4 |
| My Cloud Pro Series PR4100 from WD | $40,000 (USD) | 4 |
| 3TB My Cloud Home - Personal Cloud Storage from WD | $40,000 (USD) | 4 |
| 3TB My Cloud Home Personal Cloud from WD - firmware version 8.xx.xx-xxx (Beta) | $45,000 (USD) | 5 |

“

**WARNING**: These are competitions among hackers, not vendors. There are many other factors that go into these results, such as restrictions on exploit vectors. **Declaring something un-pwnable is a great way to get pwned.**

# We discovered several vulnerabilities but could not escalate to full RCE.


October 25, 2021: Vulnerability reports by GovTech

December 1, 2021: Confirmation and bounty award by Synology

December 12, 2021: Estimated patch timeline of March 2022 by Synology

December 16, 2021: Notification about upcoming ShmooCon talk by GovTech

February 25, 2022: CVEs issued by Synology

# Synology pwned in 2020 and had 35+ CVEs in 2021; why no pwn in 2021?

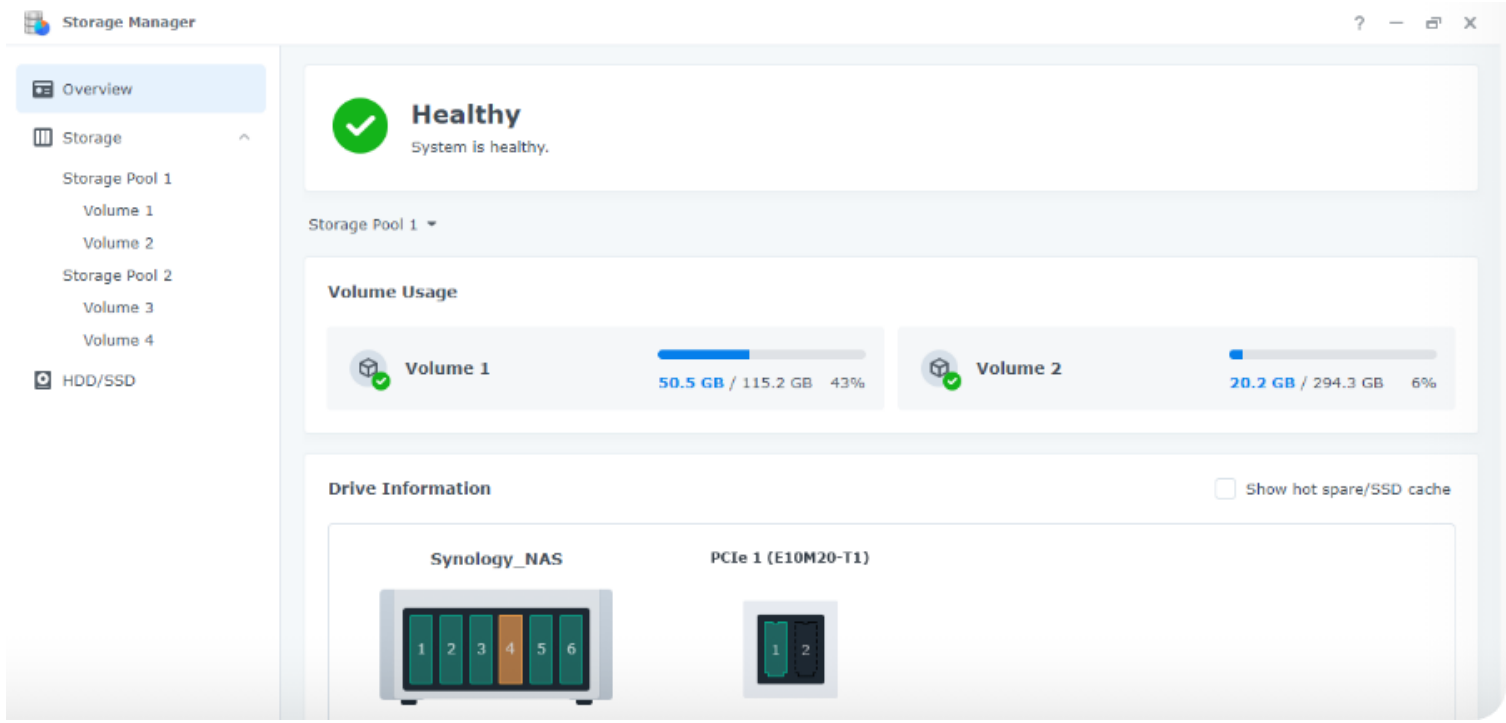
In the final entry of the contest, the STARLabs team returned to target the Synology DiskStation DS418Play NAS. They combined a race condition and an Out-Of-Bounds (OOB) Read to get a root shell on the device. This successful demonstration earned them $20,000 and 2 Master of Pwn points.


*Figure 5 - The STARLabs team observes the ZDI Bug Extraction Crew demonstrate their root shell on the Synology NAS*

| # | CVE ID | CWE ID | # of Exploits | Vulnerability Type(s) | Publish Date | Update Date | Score | Gained Access Level | Access | Complexity | Authentication | Conf. | Integ. | Avail. |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | CVE-2021-34812 | 798 | | +Info | 2021-06-18 | 2021-06-24 | 5.0 | None | Remote | Low | Not required | Partial | None | None |
| Use of hard-coded credentials vulnerability in php component in Synology Calendar before 2.4.0-0761 allows remote attackers to obtain sensitive information via unspecified vectors. | | | | | | | | | | | | | | |
| 2 | CVE-2021-34811 | 918 | | | 2021-06-18 | 2021-06-23 | 4.0 | None | Remote | Low | ??? | Partial | None | None |
| Server-Side Request Forgery (SSRF) vulnerability in task management component in Synology Download Station before 3.8.16-3566 allows remote authenticated users to access intranet resources via unspecified vectors. | | | | | | | | | | | | | | |
| 3 | CVE-2021-34810 | 269 | | Exec Code | 2021-06-18 | 2021-06-24 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Improper privilege management vulnerability in cgi component in Synology Download Station before 3.8.16-3566 allows remote authenticated users to execute arbitrary code via unspecified vectors. | | | | | | | | | | | | | | |
| 4 | CVE-2021-34809 | 77 | | Exec Code | 2021-06-18 | 2021-06-24 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Improper neutralization of special elements used in a command ('Command Injection') vulnerability in task management component in Synology Download Station before 3.8.16-3566 allows remote authenticated users to execute arbitrary code via unspecified vectors. | | | | | | | | | | | | | | |
| 5 | CVE-2021-34808 | 918 | | | 2021-06-18 | 2021-06-23 | 5.0 | None | Remote | Low | Not required | Partial | None | None |
| Server-Side Request Forgery (SSRF) vulnerability in cgi component in Synology Media Server before 1.8.3-2881 allows remote attackers to access intranet resources via unspecified vectors. | | | | | | | | | | | | | | |
| 6 | CVE-2021-33184 | 918 | | | 2021-06-01 | 2021-06-10 | 4.0 | None | Remote | Low | ??? | Partial | None | None |
| Server-Side request forgery (SSRF) vulnerability in task management component in Synology Download Station before 3.8.15-3563 allows remote authenticated users to read arbitrary files via unspecified vectors. | | | | | | | | | | | | | | |
| 7 | CVE-2021-33183 | 22 | | Dir. Trav. | 2021-06-01 | 2021-06-10 | 3.6 | None | Local | Low | Not required | Partial | Partial | None |
| Improper limitation of a pathname to a restricted directory ('Path Traversal') vulnerability container volume management component in Synology Docker before 18.09.0-0515 allows local users to read or write arbitrary files via unspecified vectors. | | | | | | | | | | | | | | |
| 8 | CVE-2021-33182 | 22 | | Dir. Trav. | 2021-06-01 | 2021-06-09 | 4.0 | None | Remote | Low | ??? | Partial | None | None |
| Improper limitation of a pathname to a restricted directory ('Path Traversal') vulnerability in PDF Viewer component in Synology DiskStation Manager (DSM) before 6.2.4-25553 allows remote authenticated users to read limited files via unspecified vectors. | | | | | | | | | | | | | | |
| 9 | CVE-2021-33181 | 918 | | | 2021-06-01 | 2021-06-10 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Server-Side Request Forgery (SSRF) vulnerability in webapi component in Synology Video Station before 2.4.10-1632 allows remote authenticated users to send arbitrary request to intranet resources via unspecified vectors. | | | | | | | | | | | | | | |
| 10 | CVE-2021-33180 | 89 | | Exec Code Sql | 2021-06-01 | 2021-06-09 | 7.5 | None | Remote | Low | Not required | Partial | Partial | Partial |
| Improper neutralization of special elements used in an SQL command ('SQL Injection') vulnerability in cgi component in Synology Media Server before 1.8.1-2876 allows remote attackers to execute arbitrary SQL commands via unspecified vectors. | | | | | | | | | | | | | | |
| 11 | CVE-2021-31439 | 122 | | Exec Code | 2021-05-21 | 2021-05-27 | 5.8 | None | Local Network | Low | Not required | Partial | Partial | Partial |
| This vulnerability allows network-adjacent attackers to execute arbitrary code on affected installations of Synology DiskStation Manager. Authentication is not required to exploit this vulnerablity. The specific flaw exists within the processing of DSI structures in Netatalk. The issue results from the lack of proper validation of the length of user-supplied data prior to copying it to a heap-based buffer. An attacker can leverage this vulnerability to execute code in the context of the current process. Was ZDI-CAN-12326. | | | | | | | | | | | | | | |
| 12 | CVE-2021-29092 | 434 | | Exec Code | 2021-06-01 | 2021-06-09 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Unrestricted upload of file with dangerous type vulnerability in file management component in Synology Photo Station before 6.8.14-3500 allows remote authenticated users to execute arbitrary code via unspecified vectors. | | | | | | | | | | | | | | |
| 13 | CVE-2021-29091 | 22 | | Dir. Trav. | 2021-06-02 | 2021-06-10 | 4.0 | None | Remote | Low | ??? | None | Partial | None |
| Improper limitation of a pathname to a restricted directory ('Path Traversal') vulnerability in file management component in Synology Photo Station before 6.8.14-3500 allows remote authenticated users to write arbitrary files via unspecified vectors. | | | | | | | | | | | | | | |
| 14 | CVE-2021-29090 | 89 | | Exec Code Sql | 2021-06-02 | 2021-06-10 | 9.0 | None | Remote | Low | ??? | Complete | Complete | Complete |
| Improper neutralization of special elements used in an SQL command ('SQL Injection') vulnerability in PHP component in Synology Photo Station before 6.8.14-3500 allows remote authenticated users to execute arbitrary SQL command via unspecified vectors. | | | | | | | | | | | | | | |

# Technical Review

# Synology released DSM v7 in June 2021 – first major release in 5 years.



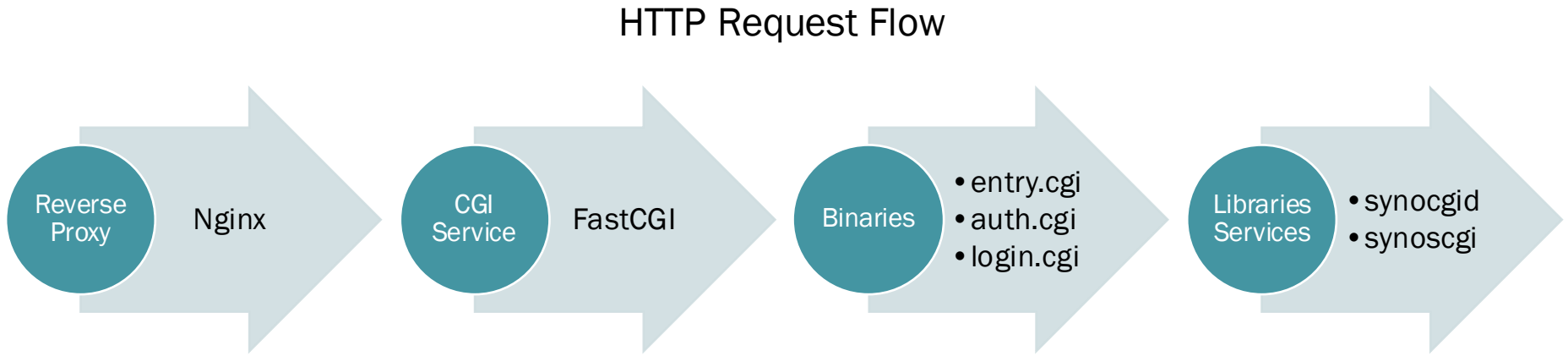

Source: Synology

# A lot of research was done by Qian Chen of Legendsec Codesafe Team.

| | |
|---|---|
| Common Services | • nginx<br>• smbd<br>• nmbd |
| Custom Services | • findhostd<br>• synosnmpcd<br>• synorelayd |

Reference: Qian Chen, “A Journey into Synology NAS”

# A lot of research was done by Qian Chen of Legendsec Codesafe Team.




Reference: Qian Chen, “A Journey into Synology NAS”

# The web binaries are split between individual CGIs or web API libraries.

```
administrator@syno:/$ ls /usr/syno/synoman/**/*.cgi
/usr/syno/synoman/DSFile/queryWebdav.cgi
/usr/syno/synoman/oauth/login_action.cgi
/usr/syno/synoman/oauth/oauth_login.cgi
/usr/syno/synoman/oauth/oauth_test.cgi
/usr/syno/synoman/oauth/oauth_token.cgi
/usr/syno/synoman/sharing/redirect.cgi
/usr/syno/synoman/sharing/sharing.cgi
/usr/syno/synoman/webapi/auth.cgi
/usr/syno/synoman/webapi/_____________________________________
/usr/syno/synoman/webapi/entry.cgi
/usr/syno/synoman/webapi/otp.cgi
/usr/syno/synoman/webman/authenticate.cgi
/usr/syno/synoman/webman/error.cgi
/usr/syno/synoman/webman/index.cgi
/usr/syno/synoman/webman/info.cgi
/usr/syno/synoman/webman/login.cgi
/usr/syno/synoman/webman/logout.cgi
/usr/syno/synoman/webman/pingpong.cgi
/usr/syno/synoman/webman/search.cgi
/usr/syno/synoman/webman/search_result.cgi
```

```
administrator@syno:/$ ls /usr/syno/synoman/webapi/lib/*.so
/usr/syno/synoman/webapi/lib/libCoreTFTP.so
/usr/syno/synoman/webapi/lib/libHardware.so
/usr/syno/synoman/webapi/lib/libNotification.so
/usr/syno/synoman/webapi/lib/libS2SClientJob.so
/usr/syno/synoman/webapi/lib/libS2SClient.so
/usr/syno/synoman/webapi/lib/libS2SServerPair.so
/usr/syno/synoman/webapi/lib/libS2SServer.so
/usr/syno/synoman/webapi/lib/libStorage.so
/usr/syno/synoman/webapi/lib/libwebapi-Authentication.so
/usr/syno/synoman/webapi/lib/libwebapi-Bond.so
/usr/syno/synoman/webapi/lib/libwebapi-CurrentConnection.so
/usr/syno/synoman/webapi/lib/libwebapi-DataCollect.so
/usr/syno/synoman/webapi/lib/libwebapi-Ethernet.so
/usr/syno/synoman/webapi/lib/libwebapi-IPv6Router.so
/usr/syno/synoman/webapi/lib/libwebapi-ipv6.so
/usr/syno/synoman/webapi/lib/libwebapi-MacClone.so
/usr/syno/synoman/webapi/lib/libwebapi-Network-Interface.so
/usr/syno/synoman/webapi/lib/libwebapi-Network.so
/usr/syno/synoman/webapi/lib/libwebapi-OVS.so
/usr/syno/synoman/webapi/lib/libwebapi-PPPoE.so
```

# The web APIs are mapped with .lib definitions to the matching libraries.

```
"SYNO.Core.Desktop.JSUIString": {
    "appPriv": "",
    "authLevel": 0,
    "disableSocket": false,
    "lib": "lib/SYNO.Core.Desktop.so",
    "maxVersion": 1,
    "methods": {
        "1": [
            {
                "getjs": {
                    "allowDemo": true,
                    "allowDownload": true,
                    "cgiProcReusable": true,
                    "grantByUser": false,
                    "grantable": false,
                    "systemdSlice": ""
                }
            }
        ]
    },
    "minVersion": 1,
    "priority": -10,
    "socket": "",
    "socketConnTimeout": 600
},
```

/usr/syno/synoman/webapi/SYNO.Core.Desktop.lib

```
unsigned __int64 __fastcall SYNO::Core::Desktop::JSUIString_v1(SYNO::Core::Desktop *this,
{
  void *v3; // rdi
  char v5[32]; // [rsp+20h] [rbp-1158h] BYREF
  char v6[32]; // [rsp+40h] [rbp-1138h] BYREF
  char v7[32]; // [rsp+60h] [rbp-1118h] BYREF
  void *v8[2]; // [rsp+80h] [rbp-10F8h] BYREF
  __int64 v9; // [rsp+90h] [rbp-10E8h] BYREF
  void *v10[2]; // [rsp+A0h] [rbp-10D8h] BYREF
  __int64 v11; // [rsp+B0h] [rbp-10C8h] BYREF
  void *v12; // [rsp+C0h] [rbp-10B8h] BYREF
  _BYTE v13[16]; // [rsp+D0h] [rbp-10A8h] BYREF
  void *v14[2]; // [rsp+E0h] [rbp-1098h] BYREF
  char v15[16]; // [rsp+F0h] [rbp-1088h] BYREF
  void *v16; // [rsp+100h] [rbp-1078h] BYREF
  void *v17; // [rsp+108h] [rbp-1070h]
  __int64 v18; // [rsp+110h] [rbp-1068h] BYREF
  __int64 v19; // [rsp+118h] [rbp-1060h] BYREF
  char v20[4104]; // [rsp+130h] [rbp-1048h] BYREF
  unsigned __int64 v21; // [rsp+1138h] [rbp-40h]

  v21 = __readfsqword(0x28u);
  memset(v20, 0, 0x1000uLL);
  Json::Value::Value((Json::Value *)v6, "enu");
  v16 = &v18;
  sub_BA20(&v16, "lang");
  SYNO::APIRequest::GetParam(v7, this, &v16, v6);
  Json::Value::asString[abi:cxx11](v8, v7);
  Json::Value::~Value((Json::Value *)v7);
  if ( v16 != &v18 )
    operator delete(v16);
  Json::Value::~Value((Json::Value *)v6);
  Json::Value::Value((Json::Value *)v6, "false");
  v16 = &v18;
  sub_BA20(&v16, "login_only");
  SYNO::APIRequest::GetParam(v7, this, &v16, v6);
  Json::Value::asString[abi:cxx11](v10, v7);
  Json::Value::~Value((Json::Value *)v7);
  if ( v16 != &v18 )
    operator delete(v16);
  Json::Value::~Value((Json::Value *)v6);
```

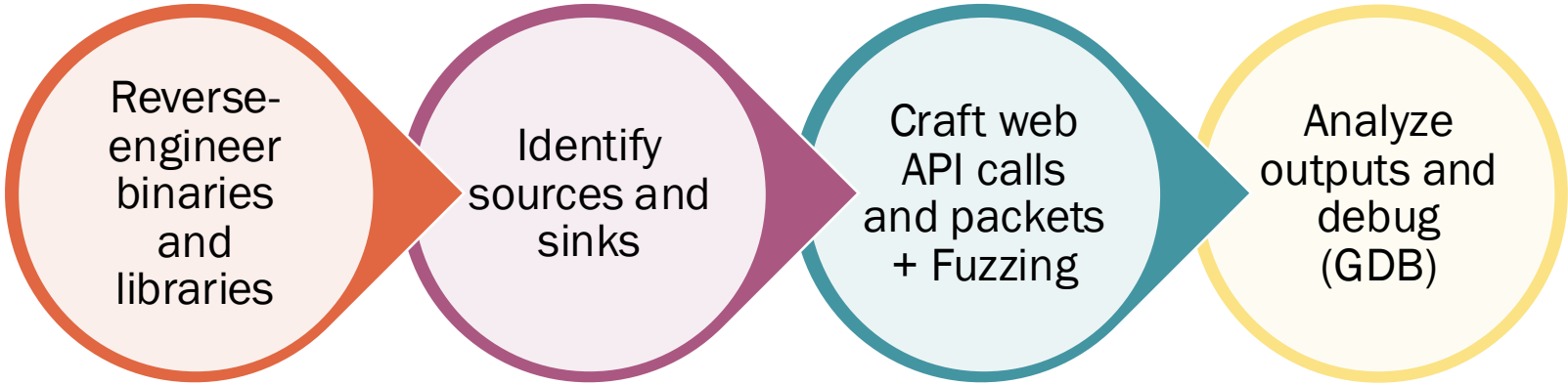
/usr/syno/synoman/webapi/lib/SYNO.Core.Desktop.so

**Request**

Pretty Raw Hex \n

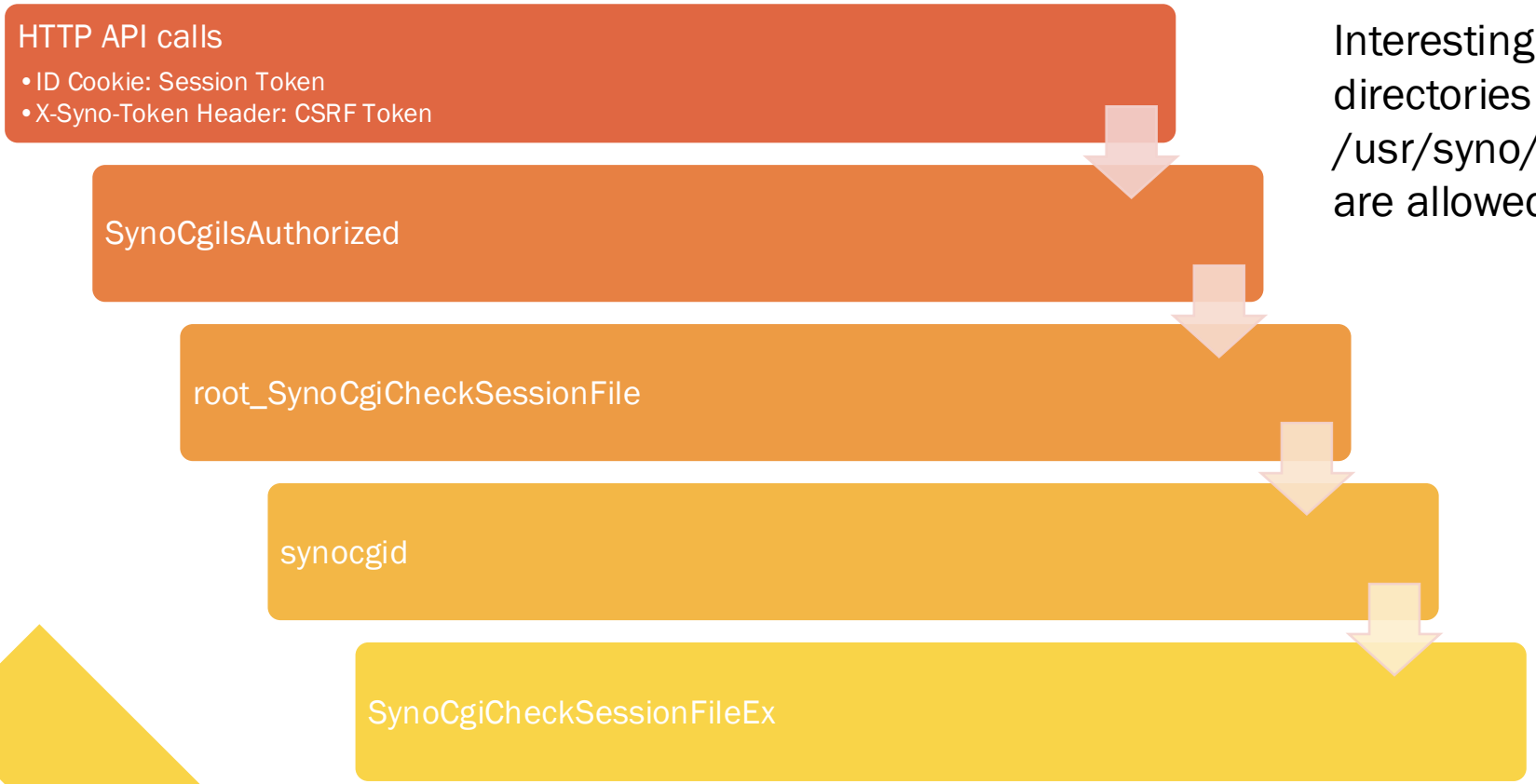
```
1 GET /webapi/entry.cgi?api=SYNO.Core.Desktop.JSUIString&method=
  getjs&lang=enu&login_only=false&version=1 HTTP/1.1
2 Host: 192.168.1.8:5000
3 X-Requested-With: XMLHttpRequest
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/92.0.4515.131
  Safari/537.36 Edg/92.0.902.73
5 Accept: */*
6 Origin: http://192.168.1.8:500
7 Referer: http://192.168.1.8:5000/
8 Accept-Encoding: gzip, deflate
9 Accept-Language: en-US,en;q=0.9
10 Connection: close
11
```

# We applied a grey-box approach including debugging on-device.



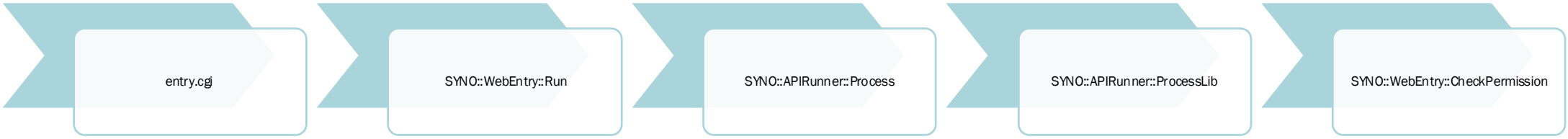
# Defensive Coding 1: Declarative Authentication and Authorization

# Authentication/Authorization checks funnels to SynoCgiIsAuthorized.




Interestingly, CGI scripts from directories listed in /usr/syno/etc.defaults/token.whitelist are allowed to bypass authentication.

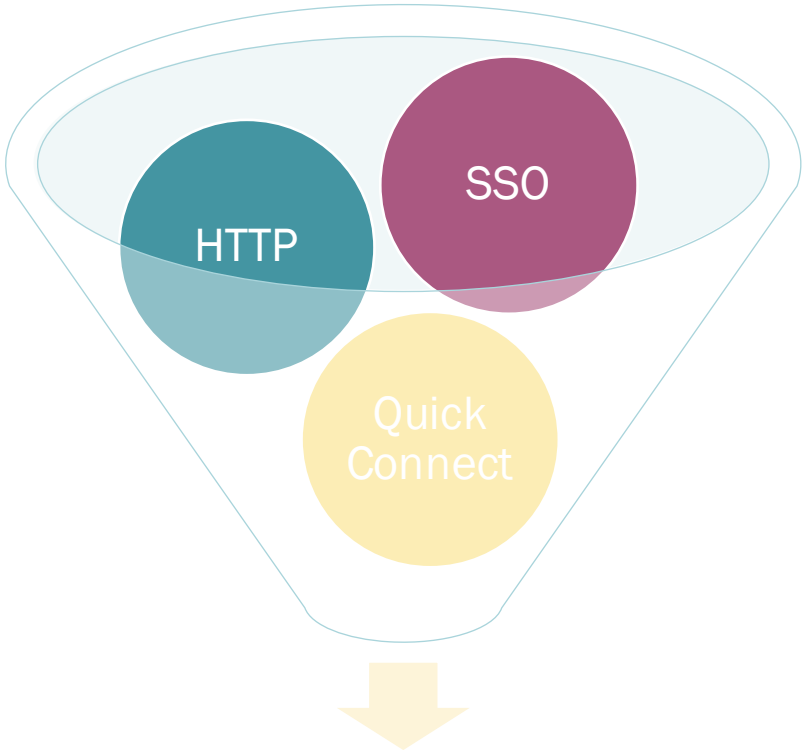
# Web API calls employ declarative authorization via the .lib definitions.




```
"SYNO.AudioPlayer.Stream": {
    "allowUser": [
        "admin.local",
        "admin.domain",
        "admin.ldap",
        "normal.local",
        "normal.domain",
        "normal.ldap"
    ],
    "appPriv": "",
    "authLevel": 1,
    "lib": "/usr/syno/synoman/webapi/lib/SYNO.AudioPlayer.so",
    "maxVersion": 2,
    "methods": {
        "2": [
            {
                "transcode": {
                    "allowDownload": true,
                    "grantByDefault": false,
                    "grantable": true
                }
            },
```

Easy to test, track, and update!

# Multiple authentication methods flow back to a single source of truth.




libsynocgi.so

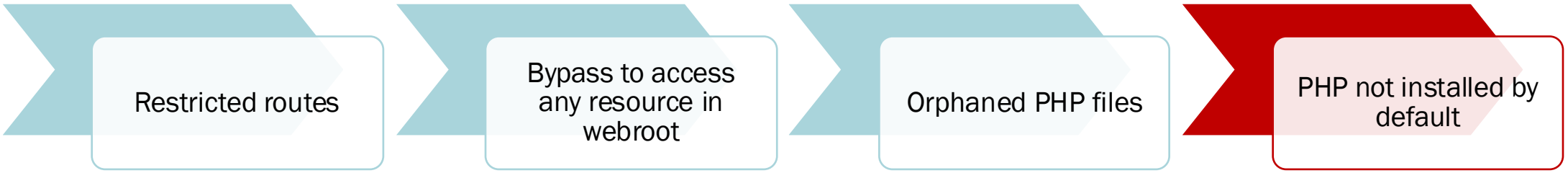
```
v12 = (const char *)SynoCgiGetCGIParam(a1, "enable_device_token", "no");
v49 = strcasecmp(v12, "yes");
v50 = -1;
memset(v62, 0, sizeof(v62));
v55 = 0LL;
*(_QWORD *)v57 = 0LL;
v54[0] = 0LL;
v54[1] = 0LL;
v54[2] = 0LL;
v54[3] = 0LL;
memset(v56, 0, sizeof(v56));
memset(v58, 0, sizeof(v58));
memset(s, 0, sizeof(s));
v60 = 0;
v61 = 0;
if ( a1 && a2 && v8 > 0 && a4 && ((*(_BYTE *)(a4 + 24) & 0xC) == 0 || *(_QWORD *)(a4 + 16)) && a5 )
{
  SynoCgiDeviceInfoGet(a1, &v55, 128LL, v57, 128LL, v54, 0.0, 64LL);
  if ( !(unsigned int)SynoCgiDeviceInfoSet(a1, &v55, 128LL, v57, v54) )
    __syslog_chk(3LL, 1LL, "%s:%d set device id failed. %s", "login.c", 1174LL, (const char *)&v55);
  if ( *(_QWORD *)a4 && *(_QWORD *)(a4 + 8) && (a7 || !memcmp("SYNOSSOLOGIN", *(const void **)(a4 + 8), 0xCuLL)) )
    goto LABEL_13;
  v13 = SynoTokenLogin(a1, s, 493LL);
  if ( v13 )
  {
    if ( a8 )
      SynoPAMLoginGetUser(a8, s, 493LL);
    v14 = strdup(s);
    SynoPAMLoginSetUser(a8, s);
LABEL_14:
    if ( v14 )
      goto LABEL_15;
    goto LABEL_61;
  }
  if ( a8 )
  {
    SynoPAMLoginSetDeviceInfo(a8, &v55, v57, v54);
    v17 = SynoPAMLogin_V3(&v50, a8);
    if ( (unsigned int)SynoPAMLoginGetState(a8, v62, 4096LL) )
      SynoCgiSetOption(a1, 13, (unsigned int)v62, v18, v19, v20);
    v21 = *(const char **)a4;
    if ( *(_QWORD *)a4 )
    {
      if ( *v21 )
        goto LABEL_35;
      if ( (unsigned int)SynoPAMLoginGetUser(a8, s, 493LL) )
      {
        v14 = strdup(s);
LABEL_36:
        if ( v14 )
        {
          v13 = sub_1E630(v17, v50, v14);
```

# Previous attacks bypassed authentication via non-web services.

- CVE-2018-1160
  - Severity: Critical
  - CVSS3 Base Score: 9.8
  - CVSS3 Vector: CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
  - Netatalk before 3.1.12 is vulnerable to an out of bounds write in dsi_opensess.c. This is due to lack of bounds checking on attacker controlled data. A remote unauthenticated attacker can leverage this vulnerability to achieve arbitrary code execution.
- CVE-2021-31439
  - Severity: Important
  - CVSS3 Base Score: 8.8
  - CVSS3 Vector: CVSS:3.0/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
  - This vulnerability allows network-adjacent attackers to execute arbitrary code on affected installations of Synology DiskStation Manager. Authentication is not required to exploit this vulnerablity. The specific flaw exists within the processing of DSI structures in Netatalk. The issue results from the lack of proper validation of the length of user-supplied data prior to copying it to a heap-based buffer. An attacker can leverage this vulnerability to execute code in the context of the current process. Was ZDI-CAN-12326.

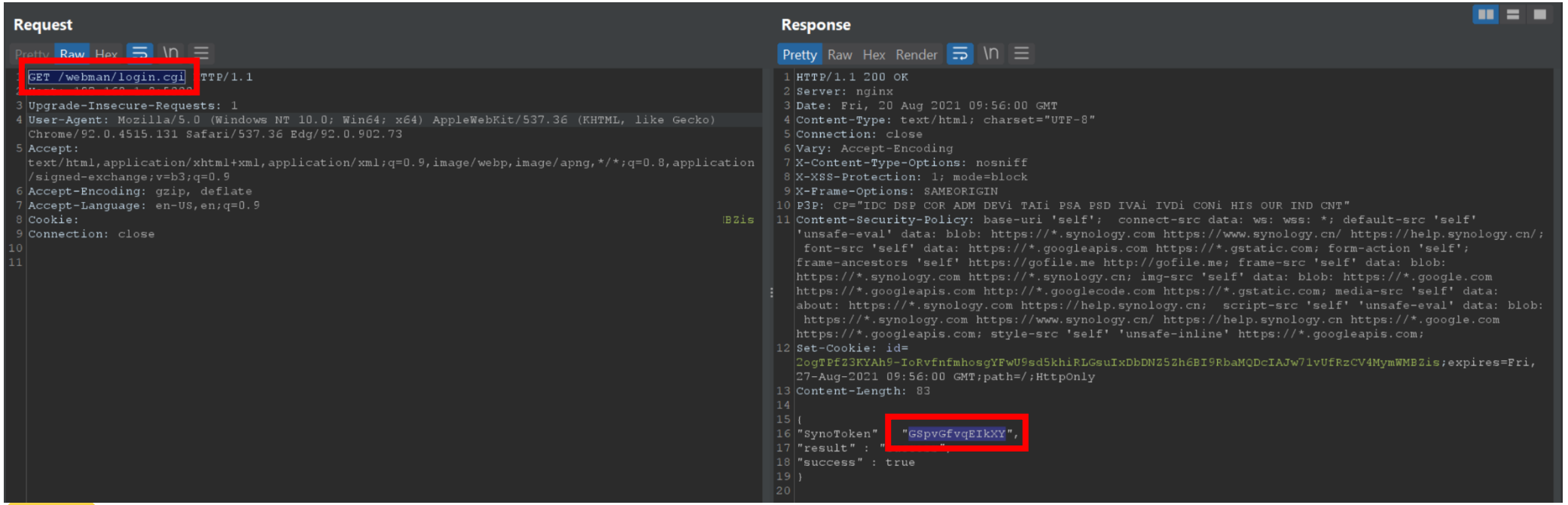
# A misconfiguration allowed us to access other resources…

…but attack surface was minimized, preventing further exploitation ☹




Several other vulnerabilities we discovered were limited to authenticated users.

# Nevertheless, the client side could still be exploited relatively easily by XSS.

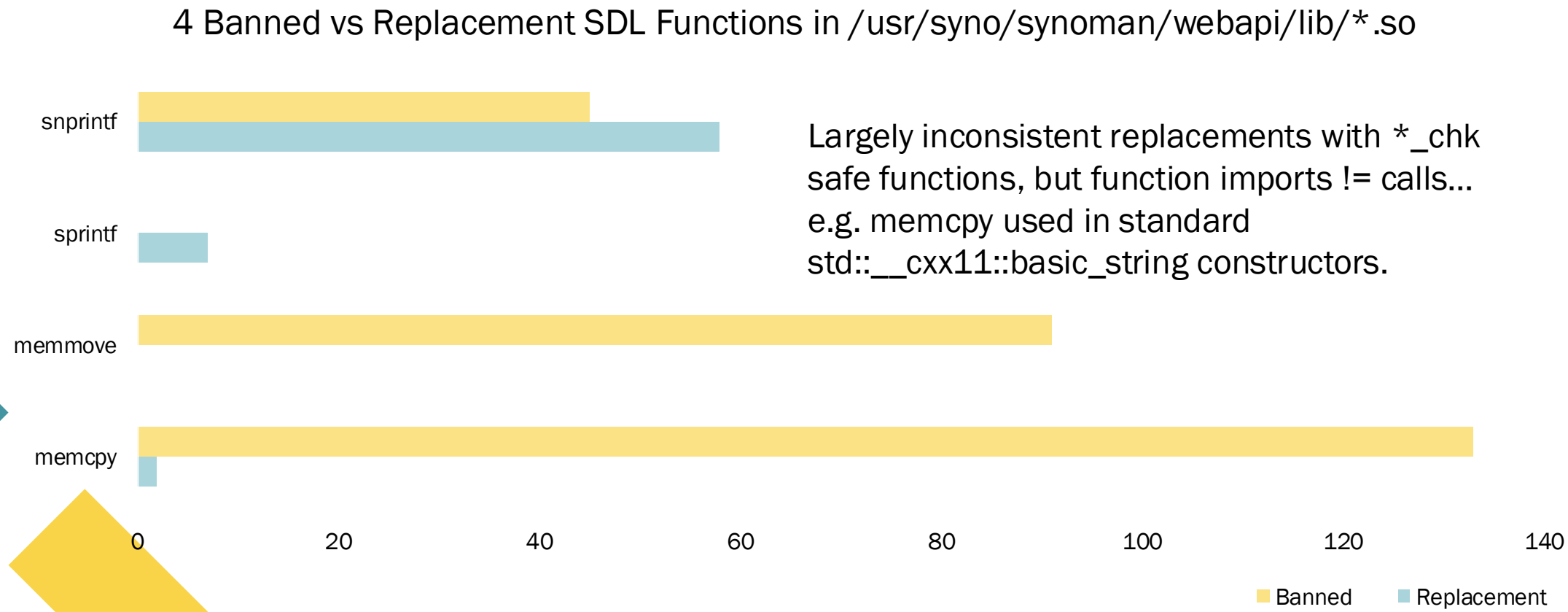



SYNO.Core.TaskScheduler create and run methods for quick code execution!

Apply declarative authentication and authorization, funnel to a single source of truth, and minimize the attack surface to authenticated APIs.

# Defensive Coding 2: Hardened Standard Library Calls

# We extracted Security Development Lifecycle banned function imports.




Largely inconsistent replacements with *_chk safe functions, but function imports != calls… e.g. memcpy used in standard std::__cxx11::basic_string constructors.

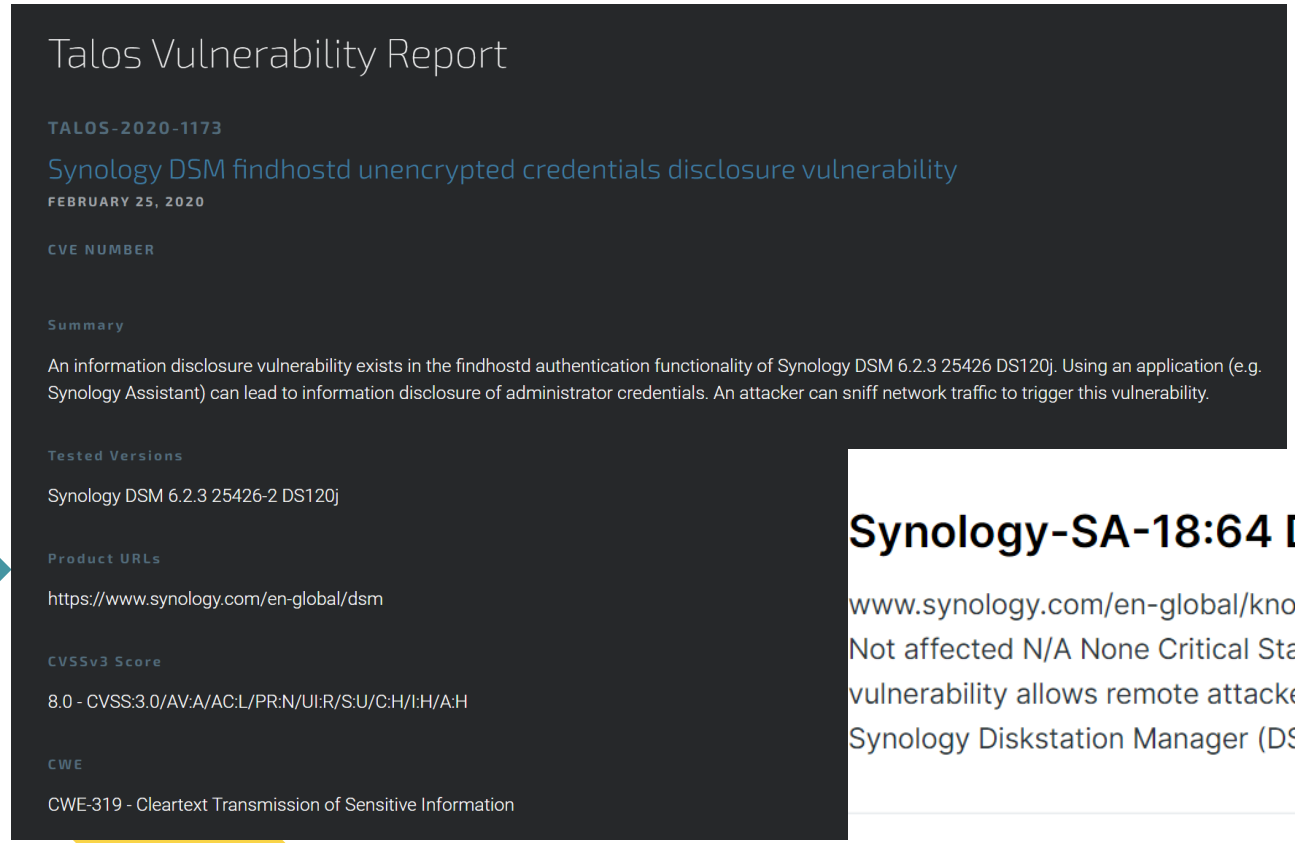
# Checks of banned function calls revealed consistent bounds-checking/resizing.

```
unsigned __int64 __fastcall sub_107F0(_QWORD *a1, _BYTE *a2, __int64 a3)
{
  size_t v3; // rbx
  _BYTE *v4; // rdx
  _BYTE *v5; // rax
  __int64 v6; // rax
  __int64 v8; // rdx
  __int64 v9; // [rsp+0h] [rbp-28h] BYREF
  unsigned __int64 v10; // [rsp+8h] [rbp-20h]

  v10 = __readfsqword(0x28u);
  if ( !a2 && a3 )
    std::__throw_logic_error("basic_string::_M_construct null not valid");
  v3 = a3 - (_QWORD)a2;
  v9 = a3 - (_QWORD)a2;
  if ( (unsigned __int64)(a3 - (_QWORD)a2) > 0xF )
  {
    v5 = (_BYTE *)std::__cxx11::basic_string<char,std::char_traits<char>,clearing_allocator<char>>::_M_create(&v9, 0LL);
    v8 = v9;
    *a1 = v5;
    a1[2] = v8;
    goto LABEL_9;
  }
  v4 = (_BYTE *)*a1;
  v5 = (_BYTE *)*a1;
  if ( v3 == 1 )
  {
    *v4 = *a2;
    v4 = (_BYTE *)*a1;
    goto LABEL_6;
  }
  if ( v3 )
  {
LABEL_9:
    memcpy(v5, a2, v3);
    v4 = (_BYTE *)*a1;
```

Common pattern of resizing suggests a wrapper function.

# Case Study: findhostd previously had memory corruption and leaks (Qian Chen).



Talos Vulnerability Report

TALOS-2020-1173

Synology DSM findhostd unencrypted credentials disclosure vulnerability

FEBRUARY 25, 2020

CVE NUMBER

Summary

An information disclosure vulnerability exists in the findhostd authentication functionality of Synology DSM 6.2.3 25426 DS120j. Using an application (e.g. Synology Assistant) can lead to information disclosure of administrator credentials. An attacker can sniff network traffic to trigger this vulnerability.

Tested Versions

Synology DSM 6.2.3 25426-2 DS120j

Product URLs

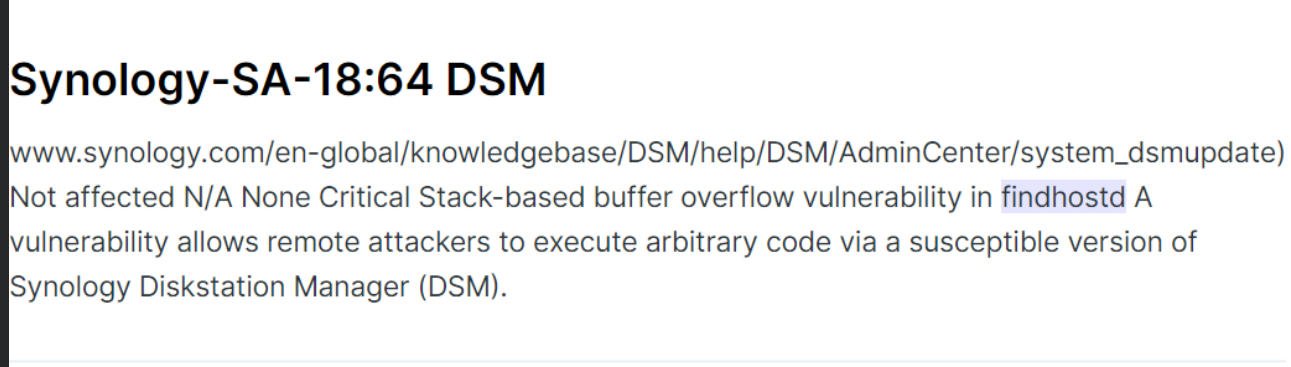
https://www.synology.com/en-global/dsm

CVSSv3 Score

8.0 - CVSS:3.0/AV:A/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H

CWE

CWE-319 - Cleartext Transmission of Sensitive Information



Synology-SA-18:64 DSM

www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_dsmupdate) Not affected N/A None Critical Stack-based buffer overflow vulnerability in findhostd A vulnerability allows remote attackers to execute arbitrary code via a susceptible version of Synology Diskstation Manager (DSM).

# Case Study: Reverse-engineering the updated protocol with a dash of OSINT.

## Changes to support encryption

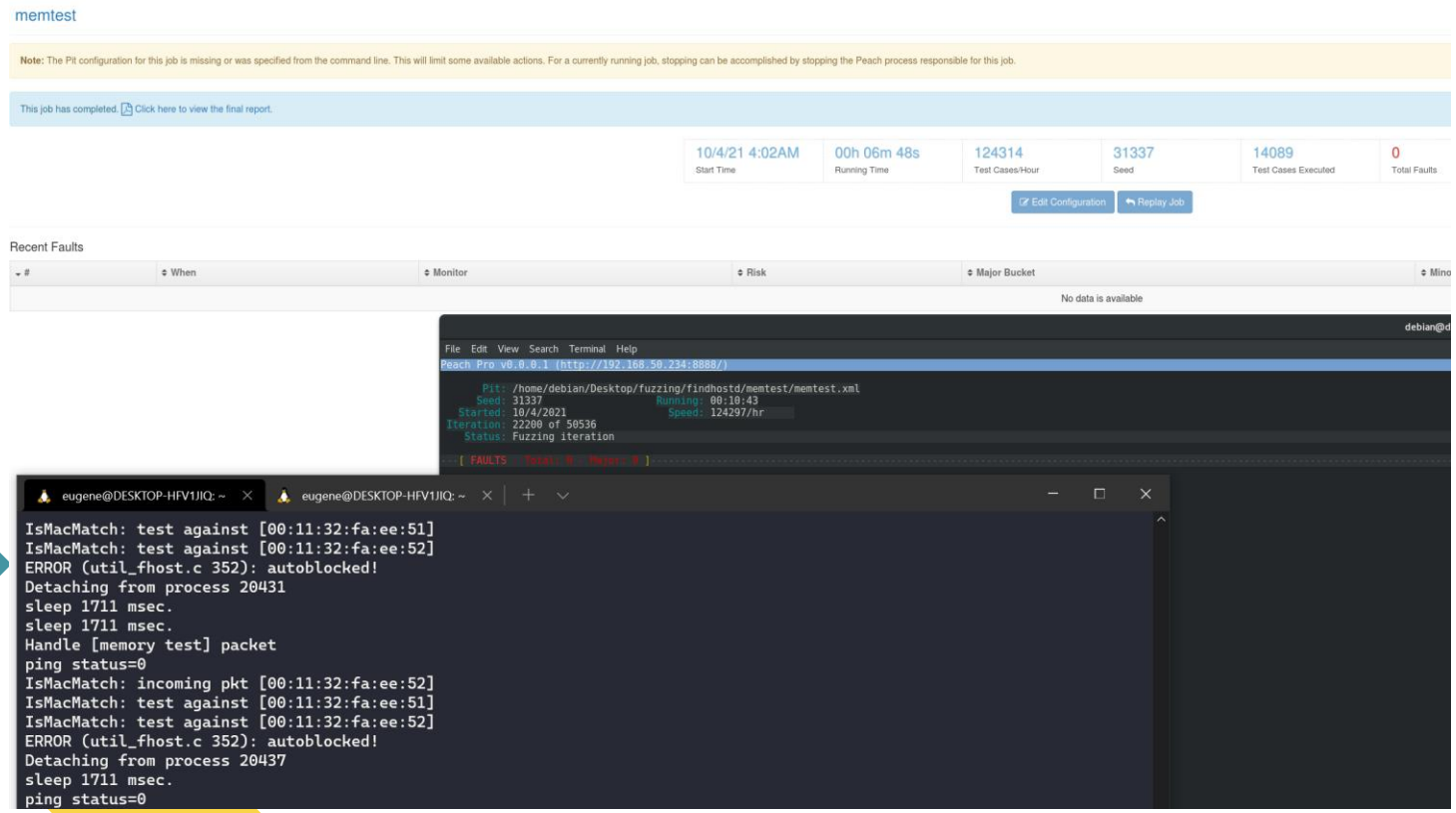
Magic bytes
\x12 4VxSYNO: plaintext packet magic bytes
\x12 4UfSYNO: encrypted packet magic bytes

New Packet IDs
\xC4: Encryption Key Packet
\xC5: Process ID Packet

Public-Key Encryption
FHOSTProcessKeyInit > crypto_box_keypair

```
/**
 * NASINFO struct
 */
struct NASINFO {
        /**
         * test connection or displayed
         * ID: none
         * Endian: Host endian
         */
        u_int32_t         iIPUnknownStatus;
        /**
         * displayed in list
         * ID: none
         * Endian: Host endian
         */
        u_int32_t         iDisplayed;
        /**
         * server name
         * ID: PKT_ID_NAME, PKT_ID_NEW_NAME
         */
        char        szName[FHOST_NASNAME_LEN+4];
        /**
         * MAC address (used to record DS MAC address)
     *  ID: PKT_ID_MAC
         */
        char        szMac[FHOST_NASNAME_LEN+4];
        /**
         * PC Source-NIC MAC address
         *  ID: PKT_ID_SRC_NIC_MAC
     */
        char            szSrcNICMac[FHOST_NASNAME_LEN+4];
```

# Case Study: Fuzzing yielded no useful results; findhostd hardened over time.




```
pktId_0 = &v14[v22];
if ( --v11 )
  continue;
if ( pLog && pLog->rgpfuncLog[2] )
{
  FHOSTLog(
    (_DWORD)pLog,
    2,
    (unsigned int)"%s:%d Maybe some virus try to hack me\n",
    (unsigned int)"packet.c",
    982,
    v9);
  return v31;
}
```

/usr/lib/libfindhost.so.1

Using Peach Fuzzer instead of Kitty (Qian Chen)

“

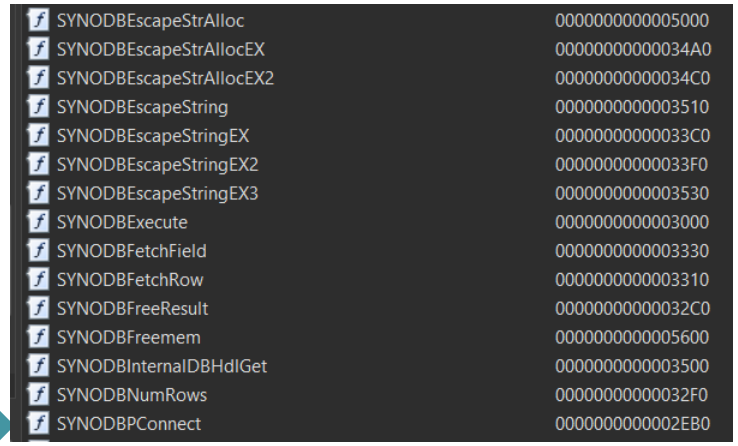
Replace memory-related standard library calls with their hardened equivalents or use code-scanning tools to enforce safe usage.

# Defensive Coding 3: Centralised Reusable Library Functions

# DSM implements shared reusable application functions extensively.

- Allows developers to secure and reuse functions at a single source. Case study: Database queries.
- The libsynodb.so library contained database functions, including string escape functions.
- These functions were both used by user facing application code to clean up input and by backend library functions.

# DB queries eventually funnelled to shared escape functions to sanitize user input.




```
36  v9 = SYNODBDatabaseTypeGet(v4);
37  v10 = SYNODBEscapeStringEX3(v9, "SELECT * FROM login WHERE uid = @SYNO:VAR AND @SYNO:VAR = '@SYNO:VAR';", v6, v8, v7);
38  v11 = (void *)v10;
39  if ( !v10 )
40  {
41    __syslog_chk(3LL, 1LL, "%s:%d SYNODBEscapeStringEX3 failed.", "SYNO.Core.SyslogClient.PersonalActivity.cpp", 402LL);
42 LABEL_14:
43    LODWORD(v8) = 0;
44    goto LABEL_7;
45  }
46  if ( (unsigned int)SYNODBExecute(v5, v10, v13) == -1 )
```

Eventually passed to SQLite's %q format string

# Other shared libraries would build upon the base libraries to extend functionality.

```
__int64 __fastcall SYNO::sharing::db::BaseDBPrivate::Insert(__int64 a1, _QWORD *a2, _QWORD *a3)
{
  _QWORD *v4; // r12
  __int64 v5; // rbp
  std::__ostream_insert<char,std::char_traits<char>>(&v35, "INSERT INTO ", 12LL);
  v8 = std::__ostream_insert<char,std::char_traits<char>>(&v35, *a2, a2[1]);
  std::__ostream_insert<char,std::char_traits<char>>(v8, " VALUES (NULL,", 14LL);
  v9 = 0LL;
  v10 = (__int64)(a3[1] - *a3) >> 5;
  if ( v10 )
  {
    while ( 1 )
    {
      SYNO::sharing::db::BaseDBPrivate::Escape((__int64)&src, a1);
      std::__ostream_insert<char,std::char_traits<char>>(v11, "'", 1LL);
```

Insert in libsynosharing.so calls Escape to sanitise input

```
_QWORD *__fastcall SYNO::sharing::db::BaseDBPrivate::Escape(_QWORD *output_buffer, __int64 a2, __int64 to_escape)
{
  int v4; // eax
  unsigned int v5; // er12
  void *v6; // rsp
  size_t v7; // rax
  unsigned int v9; // eax
  char v10[8]; // [rsp+0h] [rbp-40h] BYREF
  unsigned __int64 v11; // [rsp+8h] [rbp-38h]

  v11 = __readfsqword(0x28u);
  v4 = *(_DWORD *)(to_escape + 8);
  *output_buffer = output_buffer + 2;
  v5 = 2 * v4;
  v6 = alloca(2 * v4);
  assign_string(output_buffer, "", (__int64)"");
  if ( (unsigned int)SYNODBEscapeStringEX2(
                       0LL,
                       (__int64)v10,
                       v5,
                       *(_QWORD *)to_escape,
                       *(unsigned int *)(to_escape + 8)) == -1 )
```

Escape calls SYNODBEscapeStringEX2 from libsynodb.so

# Synology modified open-source software to reuse shared application functions.

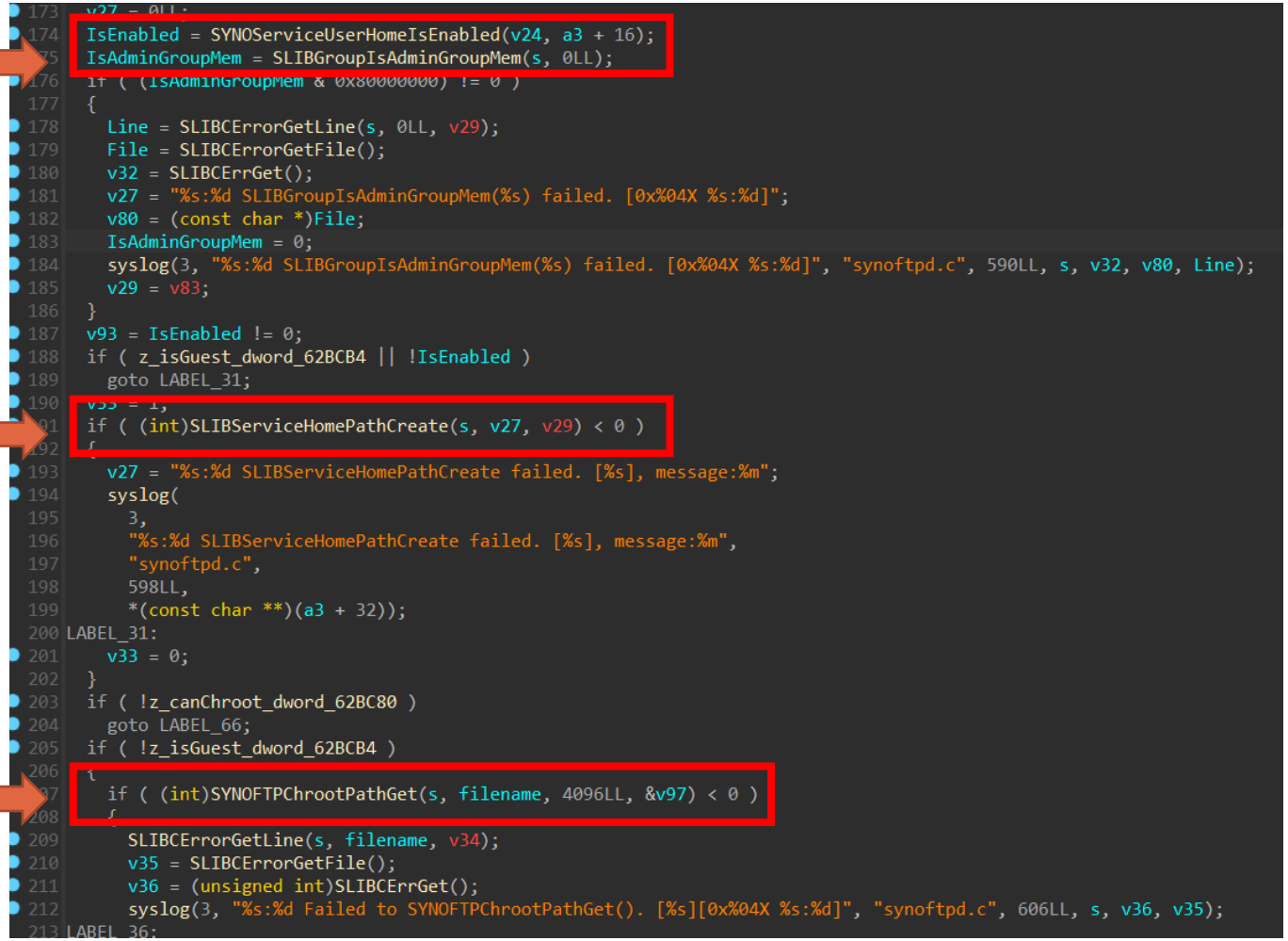
```
./ftpd

Dynamic section at offset 0x1fd60 contains 36 entries:
  Tag        Type                         Name/Value
 0x0000000000000001 (NEEDED)             Shared library: [libcrypt.so.1]
 0x0000000000000001 (NEEDED)             Shared library: [libsynotls.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynoftp.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynohacore.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynocurconn.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynorecycle.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynofileop.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynobandwidth.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynolog.so.1]
 0x0000000000000001 (NEEDED)             Shared library: [libsynonetsdk.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynocore.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libsynosdk.so.7]
 0x0000000000000001 (NEEDED)             Shared library: [libpam.so.0]
 0x0000000000000001 (NEEDED)             Shared library: [libssl.so.1.1]
 0x0000000000000001 (NEEDED)             Shared library: [libcrypto.so.1.1]
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
```

smbFTPD imports

# Synology modified open-source software to reuse shared application functions.

Original source was modified to use Synology's functions such as SYNOServiceUserHomeIsEnabled, SLIBGroupIsAdminGroupMem, SLIBServiceHomePathCreate, SYNOFTPChrootPathGet



```
IsEnabled = SYNOServiceUserHomeIsEnabled(v24, a3 + 16);
IsAdminGroupMem = SLIBGroupIsAdminGroupMem(s, 0LL);
if ( (IsAdminGroupMem & 0x80000000) != 0 )
{
  Line = SLIBCErrorGetLine(s, 0LL, v29);
  File = SLIBCErrorGetFile();
  v32 = SLIBCErrGet();
  v27 = "%s:%d SLIBGroupIsAdminGroupMem(%s) failed. [0x%04X %s:%d]";
  v80 = (const char *)File;
  IsAdminGroupMem = 0;
  syslog(3, "%s:%d SLIBGroupIsAdminGroupMem(%s) failed. [0x%04X %s:%d]", "synoftpd.c", 590LL, s, v32, v80, Line);
  v29 = v83;
}
v93 = IsEnabled != 0;
if ( z_isGuest_dword_62BCB4 || !IsEnabled )
  goto LABEL_31;
v33 = 1;
if ( (int)SLIBServiceHomePathCreate(s, v27, v29) < 0 )
{
  v27 = "%s:%d SLIBServiceHomePathCreate failed. [%s], message:%m";
  syslog(
    3,
    "%s:%d SLIBServiceHomePathCreate failed. [%s], message:%m",
    "synoftpd.c",
    598LL,
    *(const char **)(a3 + 32));
LABEL_31:
  v33 = 0;
}
if ( !z_canChroot_dword_62BC80 )
  goto LABEL_66;
if ( !z_isGuest_dword_62BCB4 )
{
  if ( (int)SYNOFTPChrootPathGet(s, filename, 4096LL, &v97) < 0 )
  {
    SLIBCErrorGetLine(s, filename, v34);
    v35 = SLIBCErrorGetFile();
    v36 = (unsigned int)SLIBCErrGet();
    syslog(3, "%s:%d Failed to SYNOFTPChrootPathGet(). [%s][0x%04X %s:%d]", "synoftpd.c", 606LL, s, v36, v35);
LABEL_36:
```

# These shared application functions called on other shared application functions.

Uses shared library functions such as SYNOUserGet and SYUNOServiceUserHomeStatusGet

Creates user's home directory by calling synouserhome with SLIBCExec to execute shell commands instead of system()

```
21  result = strcasecmp(name, "guest");
22  if ( result )
23  {
24    if ( (int)SYNOUserGet(name) >= 0 )
25
26      v10 = SYNOServiceUserHomeStatusGet(0LL, 8LL, a4);
27      switch ( v10 )
28      {
29        case 2:
30          if ( a2 )
31          {
32            if ( a3 < strlen((const char *)start) + 1 )
33            {
34              v11 = 47LL;
35              v12 = 256LL;
36              goto LABEL_16;
37            }
38            if ( (int)SLIBCStrSnprintf(a2, a3, "%s", (const char *)start) < 0 )
39              return -1;
40          }
41          if ( strncmp((const char *)start, "/var/services/homes/", 0x14uLL)
42            || !(unsigned int)sub_4DDA0((char *)start, &stat_buf) && (stat_buf.st_mode & 0xF000) == 0x4000
43            || (int)SLIBCExec("/usr/syno/bin/synouserhome", "--prepare-folder", name, 0LL, 0LL) >= 0 )
44          {
45            return 0;
46          }
```

libsynosdk.so SLIBServiceHomePathCreate

# DSM replaces dangerous functions with the secured shared application functions.

```
{
  v3 = SLIBCExecl("/usr/syno/bin/synoautoblock", 187LL, "--deny", remote_ip, 0LL);
  if ( v3 >= 0 )
  {
    v2 = "Maximum number of tries exceeded. Please contact the site manager.";
    if ( v3 )
```

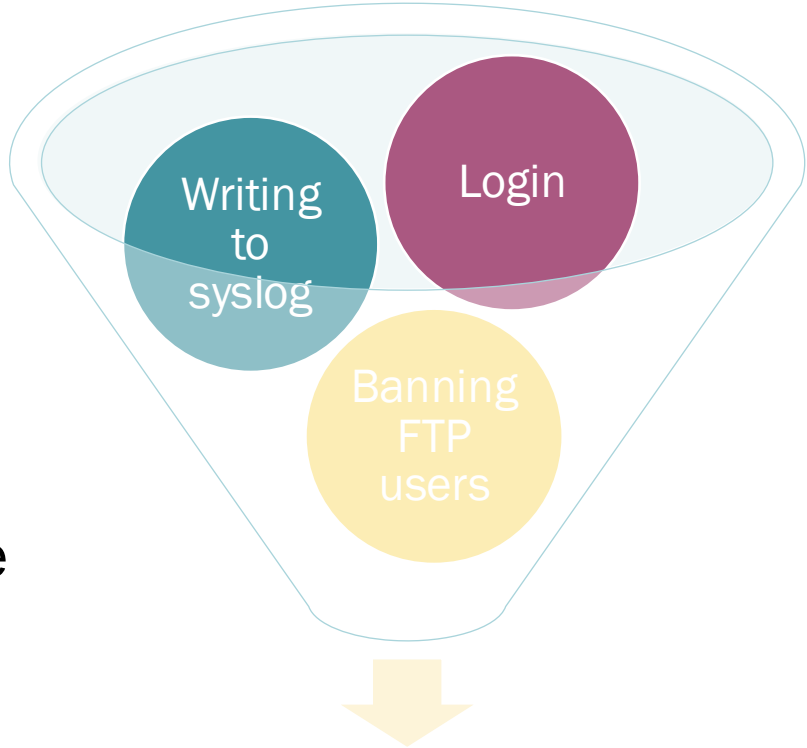
smbFTPD calls an external program, synoautoblock, to block users

```
v36[1] = (__int64)a5;
v36[0] = (__int64)"/usr/bin/convert";
v36[2] = (__int64)"-strip";
v36[3] = (__int64)"-interlace";
v36[4] = (__int64)"plane";
v36[5] = (__int64)"-quality";
v36[6] = (__int64)"25";
v36[7] = (__int64)"-define";
v36[8] = (__int64)"jpeg:extent=256KB";
__snprintf_chk(s2, 4096LL, 1LL, 4096LL, "%s/%s%d%s", path, a2, (unsigned int)a11, ".jpg");
v36[9] = (__int64)s2;
v36[10] = 0LL;
if ( !(unsigned int)SLIBCExecv("/usr/bin/convert", v36, 1LL) )
{
```

libdsm.so using SLIBCExecv to convert images into thumbnails

# Funnelling functions to the shared libraries created single source of truth to secure.

Secure the function at libsynocore to protect all calls to execute shell commands → Reduction in attack surface

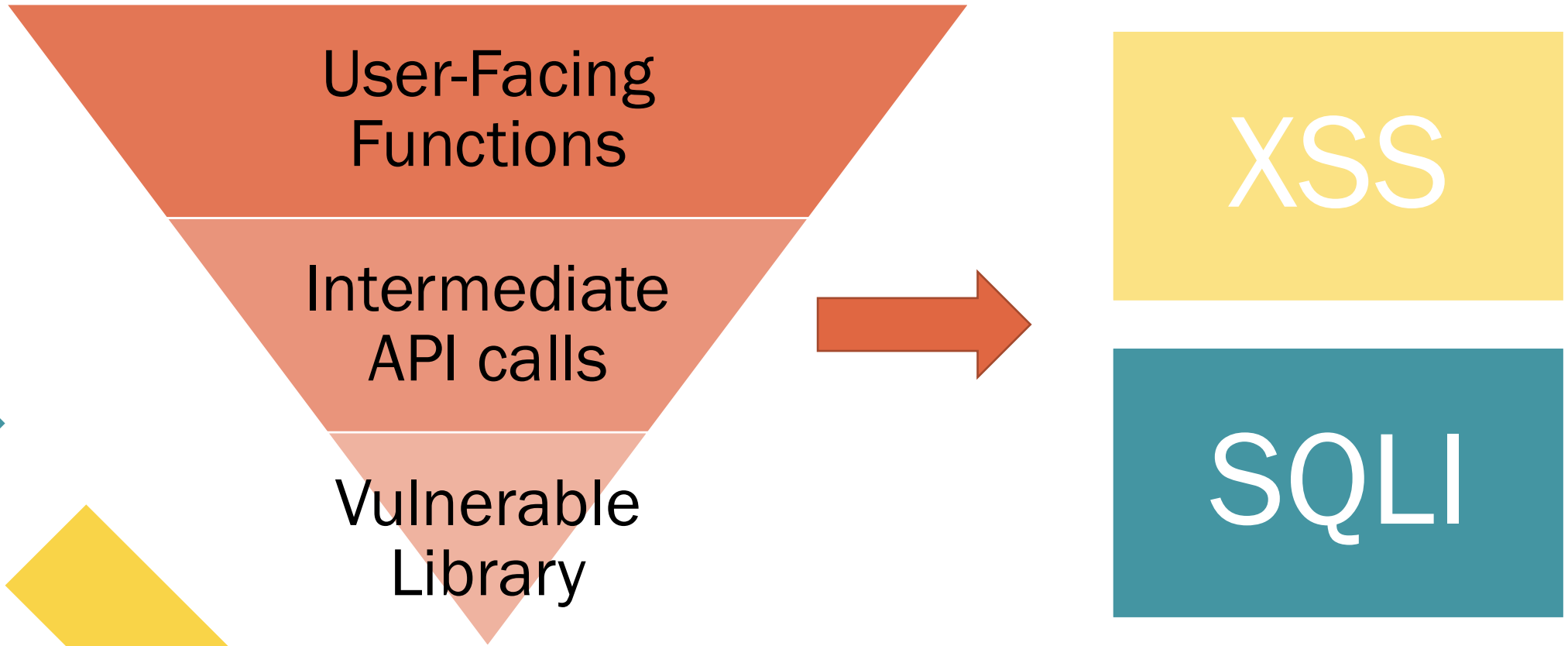


# The code created a “secure bottleneck” for all user-controlled inputs.


```
__int64 __fastcall getSQLFormatStr(__int64 a1, _QWORD *a2)
{
  const char *v2; // rax
  const char *v3; // rbp
  size_t v4; // rax
  __int64 v5; // rdx
  __int64 result; // rax

  *(_QWORD *)(a1 + 8) = 0LL;
  *(_QWORD *)a1 = a1 + 16;
  *(_BYTE *)(a1 + 16) = 0;
  v2 = (const char *)sqlite3_mprintf("%q", *a2);
  v3 = v2;
  if ( v2 )
  {
    v4 = strlen(v2);
    std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::_M_replace(
      a1,
      0LL,
      *(_QWORD *)(a1 + 8),
      v3,
      v4);
    if ( v3 )
      sqlite3_free(v3, 0LL, v5);
    result = a1;
  }
  else
  {
    __syslog_chk(3LL, 1LL, "%s:%d Can't transfer to SQL format", "oauth_utils.cpp", 135LL);
    result = a1;
  }
  return result;
}
```

# However, sometimes single sources of truth turned into single points of failure.



“

Implement critical or sensitive functions securely in a library and practice discipline in using them.

# Conclusion

# Key takeaways

Apply declarative authentication and authorization, funnel to a single source of truth, and minimize the attack surface to authenticated APIs.

Replace memory-related standard library calls with their hardened equivalents or use code-scanning tools to enforce safe usage.

Implement critical or sensitive functions securely in a library and practice discipline in using them.

# Defensive coding is not a silver bullet, but it mitigates more severe bugs → no pwn ☹

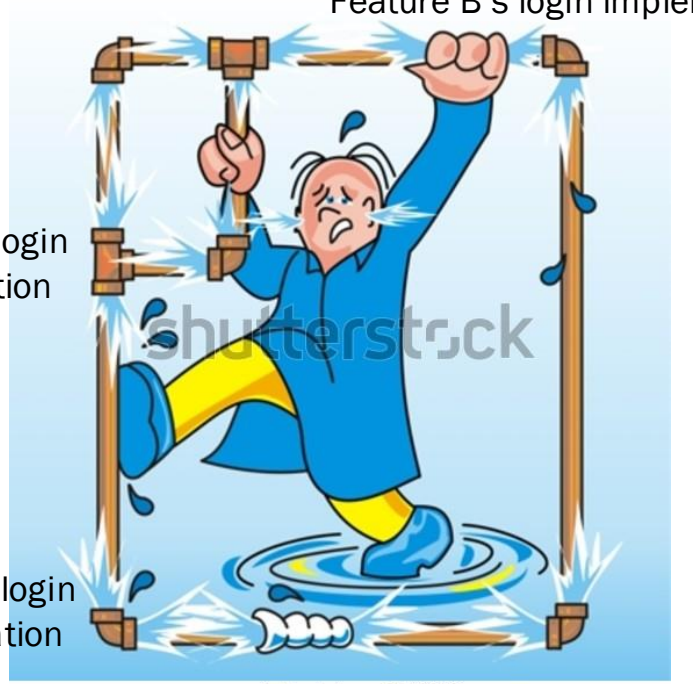
| # | CVE ID | CWE ID | # of Exploits | Vulnerability Type(s) | Publish Date | Update Date | Score | Gained Access Level | Access | Complexity | Authentication | Conf. | Integ. | Avail. |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | CVE-2021-34812 | 798 | | +Info | 2021-06-18 | 2021-06-24 | 5.0 | None | Remote | Low | Not required | Partial | None | None |
| Use of hard-coded credentials vulnerability in php component in Synology Calendar before 2.4.0-0761 allows remote attackers to obtain sensitive information via unspecified vectors. | | | | | | | | | | | | | | |
| 2 | CVE-2021-34811 | 918 | | | 2021-06-18 | 2021-06-23 | 4.0 | None | Remote | Low | ??? | Partial | None | None |
| Server-Side Request Forgery (SSRF) vulnerability in task management component in Synology Download Station before 3.8.16-3566 allows remote authenticated users to access intranet resources via unspecified vectors. | | | | | | | | | | | | | | |
| 3 | CVE-2021-34810 | 269 | | Exec Code | 2021-06-18 | 2021-06-24 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Improper privilege management vulnerability in cgi component in Synology Download Station before 3.8.16-3566 allows remote authenticated users to execute arbitrary code via unspecified vectors. | | | | | | | | | | | | | | |
| 4 | CVE-2021-34809 | 77 | | Exec Code | 2021-06-18 | 2021-06-24 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Improper neutralization of special elements used in a command ('Command Injection') vulnerability in task management component in Synology Download Station before 3.8.16-3566 allows remote authenticated users to execute arbitrary code via unspecified vectors. | | | | | | | | | | | | | | |
| 5 | CVE-2021-34808 | 918 | | | 2021-06-18 | 2021-06-23 | 5.0 | None | Remote | Low | Not required | Partial | None | None |
| Server-Side Request Forgery (SSRF) vulnerability in cgi component in Synology Media Server before 1.8.3-2881 allows remote attackers to access intranet resources via unspecified vectors. | | | | | | | | | | | | | | |
| 6 | CVE-2021-33184 | 918 | | | 2021-06-01 | 2021-06-10 | 4.0 | None | Remote | Low | ??? | Partial | None | None |
| Server-Side request forgery (SSRF) vulnerability in task management component in Synology Download Station before 3.8.15-3563 allows remote authenticated users to read arbitrary files via unspecified vectors. | | | | | | | | | | | | | | |
| 7 | CVE-2021-33183 | 22 | | Dir. Trav. | 2021-06-01 | 2021-06-10 | 3.6 | None | Local | Low | Not required | Partial | Partial | None |
| Improper limitation of a pathname to a restricted directory ('Path Traversal') vulnerability container volume management component in Synology Docker before 18.09.0-0515 allows local users to read or write arbitrary files via unspecified vectors. | | | | | | | | | | | | | | |
| 8 | CVE-2021-33182 | 22 | | Dir. Trav. | 2021-06-01 | 2021-06-09 | 4.0 | None | Remote | Low | ??? | Partial | None | None |
| Improper limitation of a pathname to a restricted directory ('Path Traversal') vulnerability in PDF Viewer component in Synology DiskStation Manager (DSM) before 6.2.4-25553 allows remote authenticated users to read limited files via unspecified vectors. | | | | | | | | | | | | | | |
| 9 | CVE-2021-33181 | 918 | | | 2021-06-01 | 2021-06-10 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Server-Side Request Forgery (SSRF) vulnerability in webapi component in Synology Video Station before 2.4.10-1632 allows remote authenticated users to send arbitrary request to intranet resources via unspecified vectors. | | | | | | | | | | | | | | |
| 10 | CVE-2021-33180 | 89 | | Exec Code Sql | 2021-06-01 | 2021-06-09 | 7.5 | None | Remote | Low | Not required | Partial | Partial | Partial |
| Improper neutralization of special elements used in an SQL command ('SQL Injection') vulnerability in cgi component in Synology Media Server before 1.8.1-2876 allows remote attackers to execute arbitrary SQL commands via unspecified vectors. | | | | | | | | | | | | | | |
| 11 | CVE-2021-31439 | 122 | | Exec Code | 2021-05-21 | 2021-05-27 | 5.8 | None | Local Network | Low | Not required | Partial | Partial | Partial |
| This vulnerability allows network-adjacent attackers to execute arbitrary code on affected installations of Synology DiskStation Manager. Authentication is not required to exploit this vulnerablity. The specific flaw exists within the processing of DSI structures in Netatalk. The issue results from the lack of proper validation of the length of user-supplied data prior to copying it to a heap-based buffer. An attacker can leverage this vulnerability to execute code in the context of the current process. Was ZDI-CAN-12326. | | | | | | | | | | | | | | |
| 12 | CVE-2021-29092 | 434 | | Exec Code | 2021-06-01 | 2021-06-09 | 6.5 | None | Remote | Low | ??? | Partial | Partial | Partial |
| Unrestricted upload of file with dangerous type vulnerability in file management component in Synology Photo Station before 6.8.14-3500 allows remote authenticated users to execute arbitrary code via unspecified vectors. | | | | | | | | | | | | | | |
| 13 | CVE-2021-29091 | 22 | | Dir. Trav. | 2021-06-02 | 2021-06-10 | 4.0 | None | Remote | Low | ??? | None | Partial | None |
| Improper limitation of a pathname to a restricted directory ('Path Traversal') vulnerability in file management component in Synology Photo Station before 6.8.14-3500 allows remote authenticated users to write arbitrary files via unspecified vectors. | | | | | | | | | | | | | | |
| 14 | CVE-2021-29090 | 89 | | Exec Code Sql | 2021-06-02 | 2021-06-10 | 9.0 | None | Remote | Low | ??? | Complete | Complete | Complete |
| Improper neutralization of special elements used in an SQL command ('SQL Injection') vulnerability in PHP component in Synology Photo Station before 6.8.14-3500 allows remote authenticated users to execute arbitrary SQL command via unspecified vectors. | | | | | | | | | | | | | | |

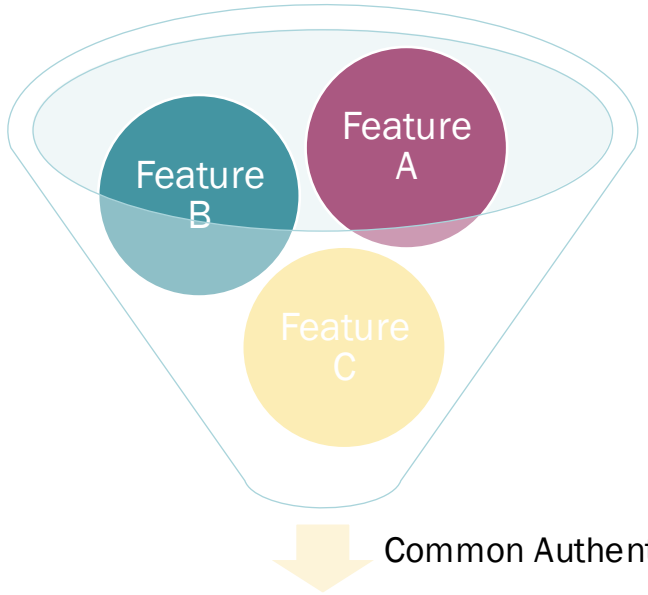
# More features doesn't always have to mean a proportionally larger attack surface.

Feature B's login implementation

Feature C's login implementation

Feature A's login implementation




Common Authentication Library

User Authenticated

# Hacking competitions apply higher standards; do they lead to stockpiling?

**Abstract**

A vulnerability allows remote authenticated users to execute arbitrary commands via a susceptible version of DiskStation Manager (DSM).

**Affected Products**

| Product | Severity | Fixed Release Availability |
|---|---|---|
| DSM 7.0 | Important | Ongoing |
| DSM 6.2 | Important | Upgrade to 6.2.4-25556-5 or above. |

**Mitigation**

None

**Detail**

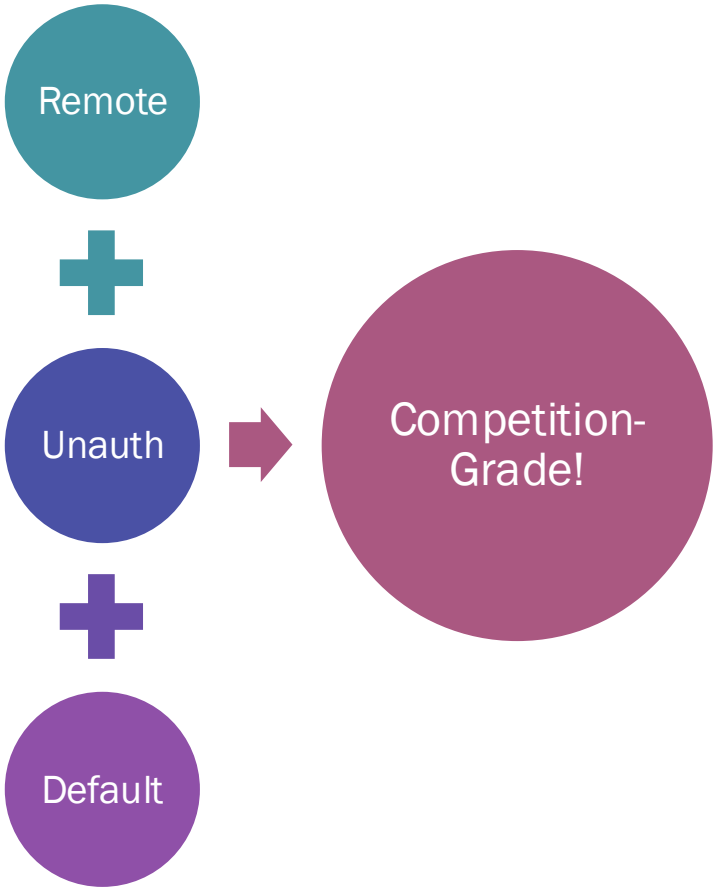
Reserved

**Acknowledgement**

Qian Chen (@cq674350529) from Codesafe Team of Legendsec at Qi'anxin Group

**Revision**

| Revision | Date | Description |
|---|---|---|
| 1 | 2022-02-22 | Initial public release. |



# Don’t be the low hanging fruit!

Hackers are great at finding soft targets.

Low impact, high volume

High impact, low volume

# Stay sharp!